

rotations.py Code Explanation

Source: *d:\Dropbox\Public Documents\PythonScripts\SRA - Original\rotations.py*

This document explains the rotation helper module used by the SRA codebase. The module has two main jobs: applying specified 3D rotations to a structured particle dataset, and estimating the bar orientation from a 2D inertia tensor so the simulated bar can be aligned with the x axis.

Executive Summary

- `rotate_model(Data, theta_z, theta_x)` rotates both positions and velocities first about z and then about x.
- `bar_along_x(Data, l_radius)` computes a 2D inertia tensor from particles inside `l_radius`, finds the bar direction from its eigenvectors, and rotates the dataset about z.
- Angles are in radians, not degrees.
- The dataset is expected to be a NumPy structured array with fields `'x'`, `'y'`, `'z'`, `'vx'`, `'vy'`, `'vz'`, and `'m'`; `rotate_model` also preserves `'tform'` if present.
- The same rotation matrices are applied to velocities as to positions, which keeps the phase-space coordinates consistent.

Function Map

<code>rotate_model</code>	General projection/viewing-angle helper. Applies $R_z(\theta_z)$, then $R_x(\theta_x)$, to position vectors and velocity vectors.
<code>bar_along_x</code>	Bar-alignment helper. Estimates the bar major-axis direction from an inertia tensor and rotates the model so that direction lies along +x or -x.

Rotation Maths Used In `rotate_model`

For each particle, the code forms a position vector p and a velocity vector v :

$$p = [x, y, z]^T, \quad v = [vx, vy, vz]^T$$

It then applies two rotations in sequence. Because the implementation uses `rot.dot(pos.T).T`, each particle column vector is transformed as follows:

$$p' = R_x(\theta_x) R_z(\theta_z) p$$

$$\mathbf{v}' = \mathbf{R}_x(\theta_x) \mathbf{R}_z(\theta_z) \mathbf{v}$$

Rotation about z:

$\cos(\theta_z)$	$-\sin(\theta_z)$	0
$\sin(\theta_z)$	$\cos(\theta_z)$	0
0	0	1

Rotation about x:

1	0	0
0	$\cos(\theta_x)$	$-\sin(\theta_x)$
0	$\sin(\theta_x)$	$\cos(\theta_x)$

The output coordinates are simply the components of the rotated vectors:

$$\mathbf{p}' = [x', y', z']^T, \quad \mathbf{v}' = [v_x', v_y', v_z']^T$$

Step-By-Step: `rotate\_model`

1. Build the z-axis rotation matrix from `theta\_z`.
2. Stack the structured position fields into an N x 3 array: columns x, y, z.
3. Stack the structured velocity fields into an N x 3 array: columns vx, vy, vz.
4. Rotate both arrays about z.
5. Build the x-axis rotation matrix from `theta\_x`.
6. Rotate the already z-rotated position and velocity arrays about x.
7. Extract the six rotated components and combine them with mass `m`.
8. If a `tform` field exists, preserve it as an eighth output column.
9. Assign the original structured dtype back to the result and return the rotated dataset.

Inertia Tensor Maths In `bar\_along\_x`

The bar-alignment function works in the x-y plane. It first keeps only particles inside a circular aperture of radius `l\_radius`:

$$R_i = \sqrt{x_i^2 + y_i^2}, \quad \text{keep particles with } R_i < l_radius$$

It then computes the following second moments from the selected particles:

$$I_{xx} = \sum x_i^2, \quad I_{yy} = \sum y_i^2, \quad I_{xy} = \sum x_i y_i$$

The code packages these into a 2D inertia-like tensor:

2D tensor used for the bar orientation:

I_{yy}	$-I_{xy}$
$-I_{xy}$	I_{xx}

Conceptually this is the 2 x 2 matrix $I = [[I_{yy}, -I_{xy}], [-I_{xy}, I_{xx}]]$. The code computes eigenvalues and eigenvectors, then selects the eigenvector associated with the smallest eigenvalue as the major-axis direction.

$$I e = \lambda e, \quad e_{\text{major}} = \text{eigenvector with } \min(\lambda)$$

If that eigenvector is $e_{\text{major}} = [e_x, e_y]^T$, the rotation angle is:

$$\theta_z = -\text{atan2}(e_y, e_x)$$

The negative sign rotates the detected bar direction back onto the x axis. After that, the function applies the same z-rotation matrix to both positions and velocities, then returns the rotated dataset and ``theta_z``.

Why Velocities Are Rotated Too

A particle's position and velocity live in the same coordinate frame. If the model is reoriented but velocities are left untouched, derived quantities such as radial velocity, angular momentum, projected kinematics, or bar-frame diagnostics would be inconsistent. Applying the same orthogonal rotation to p and v preserves lengths and angles in the rotated coordinate system.

$$|p'| = |p|, \quad |v'| = |v|, \quad p' \cdot v' = p \cdot v$$

Important Assumptions And Caveats

- The module docstring says ``theta_x` about z`, but the code correctly rotates ``theta_x`` about the x axis.
- ``bar_along_x`` only returns fields `x`, `y`, `z`, `vx`, `vy`, `vz`, and `m`; unlike ``rotate_model``, it does not preserve an optional ``tform`` field.
- Both functions rebuild a structured array by assigning ``Data.dtype`` to a ``np.column_stack`` result. This relies on the output column order and field count matching the dtype exactly.
- The bar angle can be 180 degrees ambiguous, because an eigenvector and its negative represent the same physical axis.
- ``l_radius`` should be chosen to capture the bar rather than the whole disc; otherwise the eigenvector can describe the larger disc geometry instead of the bar.

Plain-English Summary

``rotations.py`` is a small coordinate-frame utility. ``rotate_model`` changes the viewing angle of a simulation by applying standard 3D rotation matrices to every particle. ``bar_along_x`` estimates the bar's direction from the inner particle distribution and rotates the model so downstream SRA calculations can assume the bar lies along the x axis.